

Министерство образования Республики Беларусь
Учреждение образования
Белорусский государственный университет информатики и
Радиоэлектроники

Факультет информационных технологий и управления
Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №1

по курсу «Естественно-языковой интерфейс интеллектуальных
систем»

«Разработка информационно-поисковой системы.
Реализация методов оценки качества ее работы»

Вариант 7

Выполнили:

Студенты гр. 321703 Дранько И. А.
Король Г. А.

Проверил:

Крапивин Ю. Б.

Минск
2026

1. Цель работы

Освоить на практике основные принципы реализации информационно-поисковых систем и методы оценки качества их работы. В рамках варианта 7 реализована логическая (Boolean) модель поиска для документов на английском языке. Акцент варианта — **интерфейс с пользователем** как элемент ИИ: подсказки терминов, подсветка совпадений, связанные термины, загрузка документов. Система предназначена для работы в локальной вычислительной сети.

2. Структура разработанной системы

Система имеет модульную архитектуру:

1. **Модуль хранения** (`models/storage.py`) — SQLite-база документов и JSON-снимок коллекции.
2. **Модуль индексирования** (`models/index.py`) — построение инвертированного индекса, расчёт IDF и весов терминов по формулам (1.5)–(1.6).
3. **Модуль обработки запросов** (`models/search.py`) — парсер Boolean-выражений и операции AND, OR, NOT.
4. **Модуль оценки качества** (`models/metrics.py`) — метрики ROMIP (Precision, Recall, F-measure, P@k, AP, R-Precision).
5. **Элементы ИИ в интерфейсе** (`models/interface_ai.py`) — автодополнение, подсветка, связанные термины, примеры запросов.
6. **Пользовательский интерфейс** — веб (Flask) и консольный режим.

2.1. Архитектура

SQLite/JSON → Document → Токенизация → Инвертированный индекс
→ Boolean-процессор → UI

```
Lab1/  
app.py, console.py  
models/    document, index, search, metrics,  
           interface_ai, storage  
data/      documents.db, documents.json,  
           collection.py  
templates/, static/
```

Инвертированный индекс хранит для каждого термина множество идентификаторов документов (postings list). Операции AND, OR, NOT реализуются как пересечение, объединение и разность множеств.

3. Структура хранения данных

Документы хранятся в **SQLite** (`data/documents.db`):

```
documents (  
  id      INTEGER PRIMARY KEY,  
  title   TEXT NOT NULL,  
  text    TEXT NOT NULL,  
  date    TEXT  
)
```

При каждом изменении коллекции формируется JSON-снимок (`data/documents.js`).
Стартовая коллекция и экспертные оценки релевантности (`qrels`) заданы в `data/collection.py`.

В оперативной памяти после загрузки:

- `documents`: `Dict[int, Document]` — документы с токенами и частотами терминов;
- `index`: `Dict[str, Set[int]]` — инвертированный индекс;
- `df` — число документов, содержащих термин (для IDF);
- `RELEVANCE_JUDGMENTS` — `qrels` для расчёта метрик.

Через веб-раздел «Загрузка» можно добавить текст, файл `.txt`, импортировать JSON или удалить документ; индекс перестраивается автоматически.

4. Основные алгоритмы

4.1. Построение инвертированного индекса

Для каждого документа:

1. токенизация (`[a-z]+`), нижний регистр;
2. удаление стоп-слов;
3. подсчёт частоты термина в документе Q_j ;
4. добавление `doc_id` в `postings`-список каждого уникального термина;
5. обновление P_i (число документов с термином i).

Веса по методичке:

$$B_i = \log \frac{N}{P_i},$$
$$A_j = Q_j \times B_i.$$

4.2. Обработка Boolean-запроса

1. Парсинг запроса на термины и операторы AND/OR/NOT.
2. Замена терминов на postings-множества.
3. NOT — разность с полным множеством документов; AND — пересечение; OR — объединение.
4. Результат: ссылка на документ, snippet с подсветкой, список совпавших терминов и их веса.

4.3. Элементы ИИ в интерфейсе (вариант 7)

- **Автодополнение** — подсказки из словаря индекса при вводе (/api/suggest).
- **Подсветка** — слова запроса выделяются в фрагментах документов.
- **Связанные термины** — слова, часто встречающиеся в тех же документах, что и запрос.
- **Примеры запросов** — по частотным терминам коллекции.

4.4. Вычисление метрик качества

По рекомендациям ROMIP'2004:

$$\begin{aligned}\text{Precision} &= \frac{|\text{Retrieved} \cap \text{Relevant}|}{|\text{Retrieved}|}, \\ \text{Recall} &= \frac{|\text{Retrieved} \cap \text{Relevant}|}{|\text{Relevant}|}, \\ \text{F-measure} &= \frac{2 \cdot P \cdot R}{P + R}.\end{aligned}$$

Также: Precision@k, Average Precision (AP), R-Precision. Результаты выводятся таблицей и графиками (Chart.js).

5. Информация о тестовой коллекции

10 англоязычных документов (Computer Science / AI): Introduction to AI, Machine Learning Fundamentals, NLP, Computer Networks, DBMS, Information Retrieval, Deep Learning, Cybersecurity, Cloud Computing, Software Engineering.

Для 10 Boolean-запросов заданы экспертные оценки релевантности (qrels). Коллекция загружается в SQLite и доступна для расширения через интерфейс.

6. Результаты оценки качества

Таблица 1: Метрики качества поиска (ROMIP)

Запрос	P	R	F1	P@5	P@10	AP	R-P
AI OR ML	1.000	0.750	0.857	1.000	0.300	0.750	0.750
network AND protocol	1.000	1.000	1.000	1.000	0.100	1.000	1.000
database OR sql	1.000	1.000	1.000	1.000	0.100	1.000	1.000
natural language processing	1.000	1.000	1.000	1.000	0.200	1.000	1.000
cybersecurity AND encryption	1.000	1.000	1.000	1.000	0.100	1.000	1.000
cloud computing	1.000	1.000	1.000	1.000	0.100	1.000	1.000
deep learning AND neural	1.000	1.000	1.000	1.000	0.200	1.000	1.000
software engineering	0.500	1.000	0.667	0.500	0.100	0.500	0.000
IR OR boolean	0.500	1.000	0.667	0.500	0.100	1.000	1.000
ai AND learning NOT network	1.000	0.333	0.500	1.000	0.100	0.333	0.333
AVERAGE	0.900	0.908	0.869	—	—	0.858	—

Средние: Precision ≈ 0.90 , Recall ≈ 0.91 , F1 ≈ 0.87 , MAP ≈ 0.86 . Высокая точность обусловлена строгостью Boolean-модели.

7. Анализ результатов и предложения по улучшению

Сильные стороны: высокая точность; прозрачность выдачи; интеллектуальный интерфейс (подсказки, подсветка, related terms); персистентное хранение и загрузка документов; работа в локальной сети.

Недостатки: нет ранжирования; чувствительность к морфологии (нет стемминга); при изменении коллекции qrels для метрик нужно обновлять вручную.

Предложения: стемминг/лемматизация; скобки и фразы в запросах; ранжирование внутри Boolean-результата (например, по сумме весов); расширение qrels; хранение postings на диске для больших коллекций.

8. Используемые готовые компоненты

- Python 3 — язык реализации;
- Flask — веб-интерфейс, развёртывание в локальной сети (`host=0.0.0.0`);
- SQLite — хранение документов;
- Chart.js — графики метрик в браузере;
- стандартная библиотека (`re`, `collections`, `json`, `sqlite3`).

9. Запуск системы

```
pip install flask  
python3 app.py  
python3 console.py
```

Страницы: поиск, документы, загрузка, метрики, справка. Из локальной сети: <http://<IP>:5000>.

10. Заключение

Разработана ИПС с логической стратегией поиска для англоязычных документов. Реализованы хранение в SQLite, загрузка документов, веб-интерфейс с элементами ИИ (автодополнение, подсветка, связанные термины) и оценка качества по метрикам ROMIP. Система удовлетворяет требованиям варианта 7.